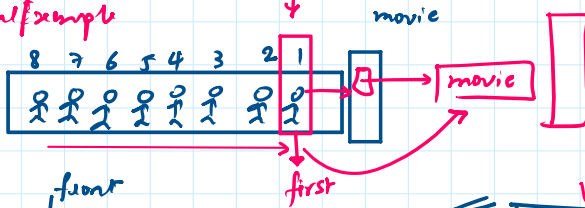


* Linear DS

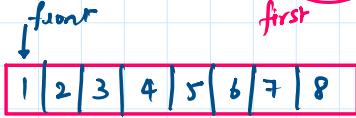
* **FIFO**: principle → First in first out.

Stack (DS) → LIFO: Last in first out

: Real example

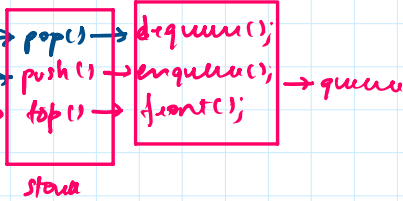


Two pointers:



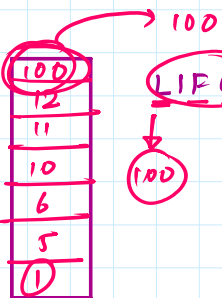
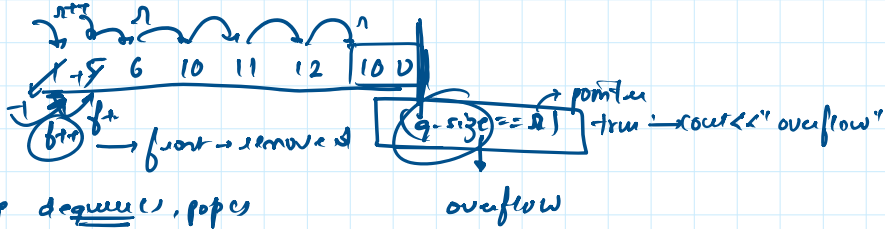
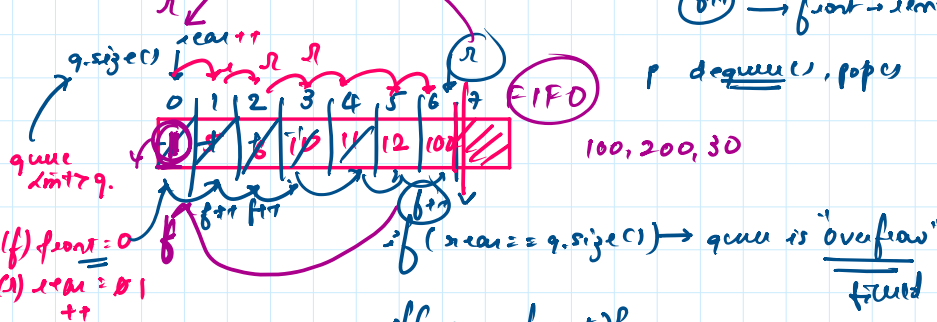
queue limit 79,

* operations



Array
Linked list
queue implement

queue.



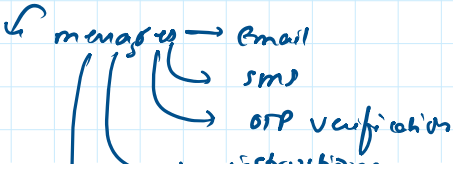
LIFO → stack

1, 5, 6, 10, 11, 12, 100

FIFO → queue
1 → 1

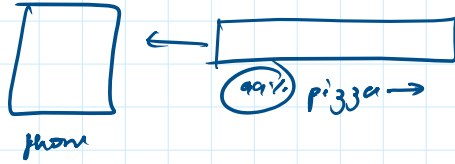


* SQS → Simple Queue System



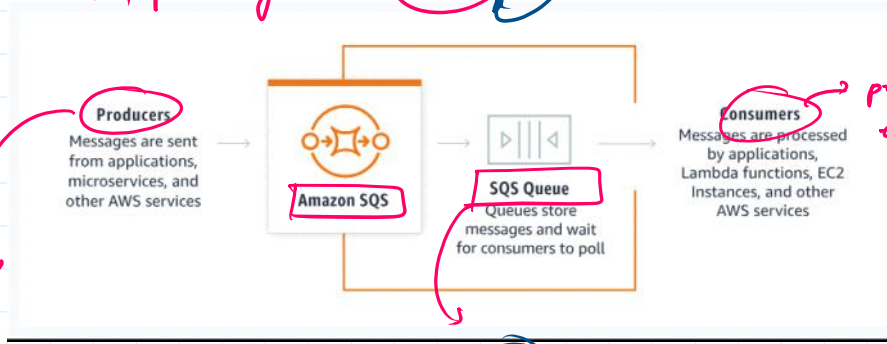
→ people queue system

sms
OTP verification
instructions
push notification
→ Zomato



SQS, priority queue, Queue → GitHub

infer sender, OTP



proper end user

System Design

S/R

* latency
* throughput
* crash the system

/s Event should run smoothly

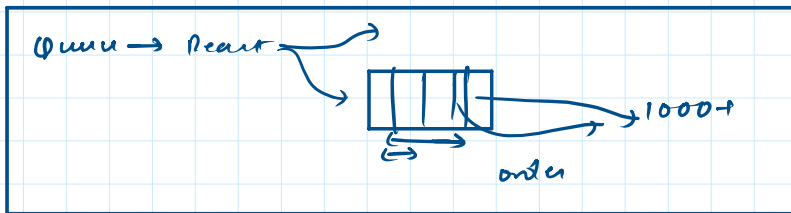
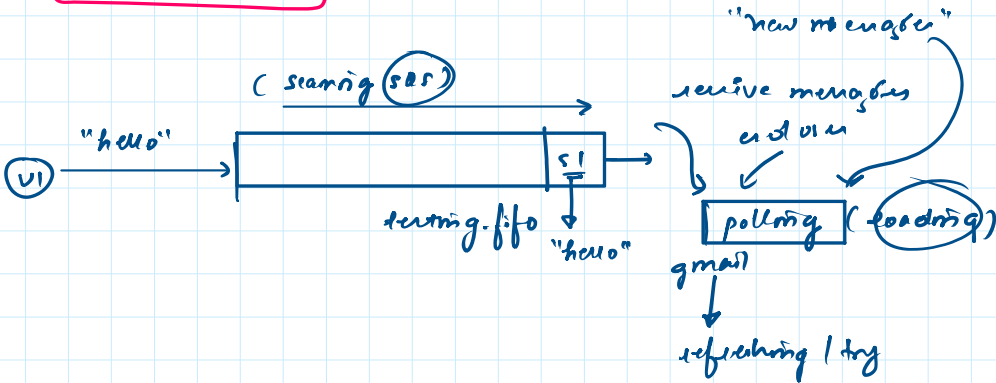
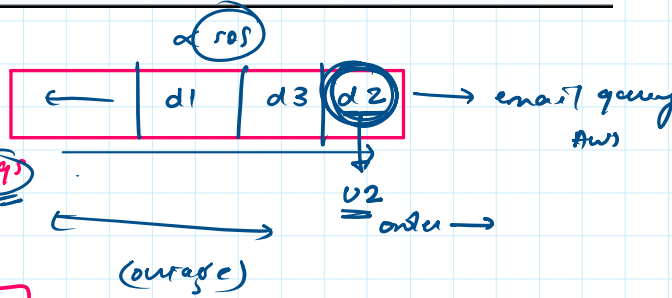
Burned Queue
RabbitMQ

Amazon's recent

million sale

iphone →

40,000 \$0.00pm

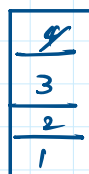


queue FIFO



pop → dequeue
= (1)

stack LIFO



pop

= (4)

pop → dequeue

= 1

1

= 4

Queue:

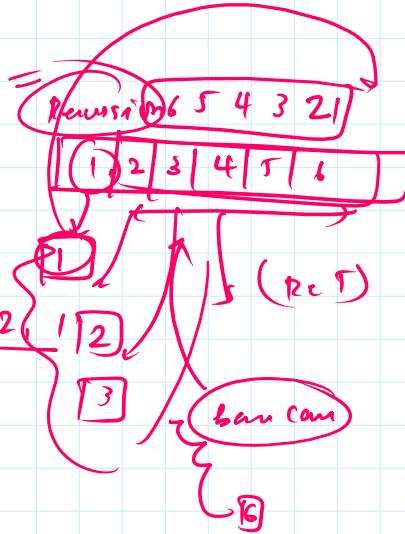
Reverse Queue / Array

1 2 3 4 5 6

stack < int >

reverse();

push(6, 5, 4, 3, 2)



ban cam

Circular Queue; priority Queue

0 1 2 3 4 5 6 7

"%"

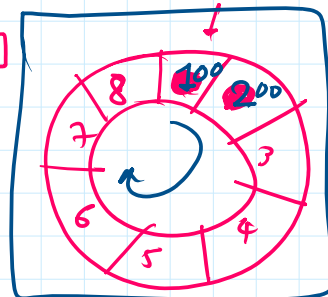
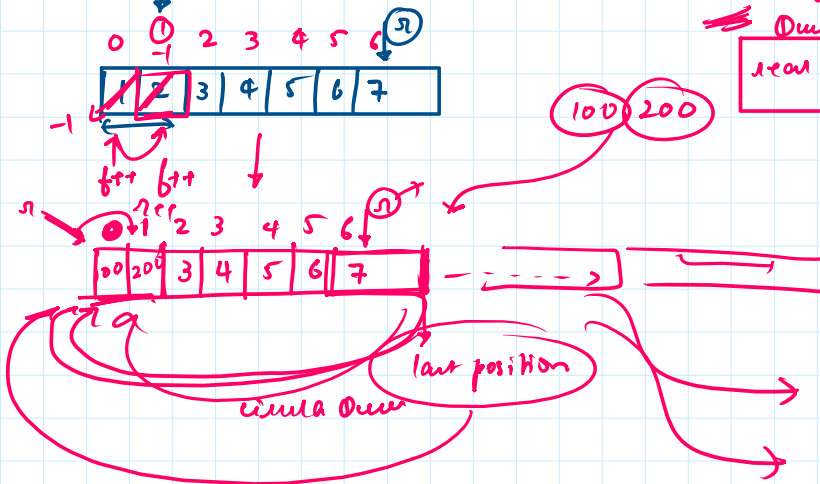
size = 6

Queue = Circular Queue

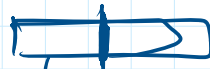
rear = (front - 1) % (size - 1)

(1 - 1) % (6 - 1)

(0) % (5) = 0



o(1) → binary search, heap sort

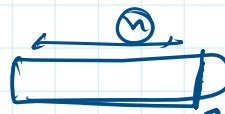


n/2

n/2 = k

n/2

log₂ n = (log₂ n)



arr[i + 1] = 5

arr

1 2 3 4 5 6

O(n)

queue < int >

q.push(1); → O(1)

for (int i = 0; i < n) {

q.push(1); $\rightarrow O(1)$

for (int i = 0; i < n) {

arr[i+1] = 5

